Open NeuroHealth Framework (StrokeAI prototype)

Author: Parameshwar

Date of Creation: 2025-11-03

Software Copyright Submission (Source Code Extract)

■Open NeuroHealth Framework — Code Export (UTF-8)
Generated: 2025-11-03T23:06:49.7609354+05:30
==========
----- BEGIN FILE: README.md -----
Open-NeuroHealth-Framework
A neuroscience ecosystem for early stroke detection, neuro-health monitoring, and open research collaboration.
\# ðŸ§ Open NeuroHealth Framework (ONF)
A unified, ethical, and open neuroscience platform integrating early neurological detection, patient care, research collaboration, and education.

\## ðŸš€ Vision
To make brain health accessible, measurable, and improvable for every human through open digital neuroscience.

\## ðŸ§© Core Modules
1\. **NeuroHealth:** AI-based stroke and neuro-symptom detection via camera, speech, and movement.
2\. **NeuroCare:** Clinical dashboard and rehabilitation tracking.
3\. **NeuroResearch:** Secure data sharing for neuroscience and BBB studies.
4\. **NeuroLearn:** Interactive neuro-learning and awareness platform.

\## ðŸ“, Project Structure
| Folder | Description |

```
|---------|-------------|
| `app/` | Mobile + web app source |
| `data/` | Sample datasets and anonymized examples |
| `models/` | AI models and algorithms |
| `research/` | Notes, experiments, and publications |
| `docs/` | Diagrams, ethics, white paper |
| `modules/` | disease-specific code (stroke features, fusion, UI) |
---
\## ðŸ”■ Status
This repository is \*\*private and under development\*\*.
Intellectual property belongs to \*\*PARAMESHWAR\*\*.
---
\## ðŸ“… Timeline
\*\*Phase 1 (3 months):\*\* Facial asymmetry + speech prototype.
\*\*Phase 2 (6 months):\*\* Multi-modal neuro-detection.
\*\*Phase 3 (12 months):\*\* Research integration and educational interface.
---
\## ðŸ§  Creator
\*\*PARAMESHWAR\*\*
NIUS Biology Fellow | Undergraduate Biotechnology Student
Email: paramn2006@gmail.com
LinkedIn: https://www.linkedin.com/in/paramn06/
```

```
Personal Website: https://www.paramn06.in/
Open NeuroHealth Framework
Version: v1.0 (Day 7 Prototype)
Copyright Â© 2025 Parameshwar
A modular AI framework for neuro-diagnostic and awareness research.
Includes stroke facial asymmetry and speech rate modules.
Developed using: Python, OpenCV, Mediapipe, NumPy, SciPy, SoundDevice
Date: November 2025
----- END FILE: README.md -----
----- BEGIN FILE: app\strokeai_demo.py -----
# ------------------------------------------------------------
#  Open NeuroHealth Framework © 2025 Parameshwar
#  Author: Parameshwar  |  Version: 1.0 (Day 7 Build)
#  Licensed under the MIT License
#  This software is an original work developed for
#  neuro-diagnostic and research awareness applications.
# ------------------------------------------------------------
# app/strokeai_demo.py
# Open NeuroHealth — StrokeAI Demo Launcher
import sys, os, subprocess, glob
PY = sys.executable
def run_strokeai():
    print("\n■ Launching StrokeAI (webcam)…\n")
    subprocess.run([PY, "modules/stroke/features/face_asymmetry.py"])
def plot_latest():
    print("\n■ Plotting latest CSV…\n")
    paths = glob.glob("data/logs/face_ai_session_*.csv")
    if not paths:
        print("■■ No CSV found in data/logs. Record once with option 1.")
        return
    latest = max(paths, key=os.path.getmtime)
    subprocess.run([PY, "tools/plot_ai.py", latest])
def cam_probe():
    print("\n■ Camera probe…\n")
    subprocess.run([PY, "tools/cam_probe.py"])
def system_check():
    print("\n■ System check…\n")
    subprocess.run([PY, "tools/system_check.py"])
```

```
def run_speech():
    print("\n■■ Speech rate recorder…\n")
    subprocess.run([PY, "modules/stroke/features/speech_rate.py"])
def fuse_face_speech():
    print("\n■ Fusing Face + Speech signals…\n")
    subprocess.run([PY, "modules/stroke/fusion/fuse_face_speech.py"])
def main():
    while True:
        print("\n=== Open NeuroHealth — StrokeAI Demo ===")
        print("1) Run StrokeAI (webcam, record CSV/JSON)")
        print("2) Plot latest session CSV")
        print("3) Camera probe")
        print("4) System check")
        print("5) Record speech (syll/sec JSON)")
        print("6) Fuse Face + Speech (risk JSON)")
        print("q) Quit")
        choice = input("Select: ").strip().lower()
        if choice == "1":
            run_strokeai()
        elif choice == "2":
            plot_latest()
        elif choice == "3":
            cam_probe()
        elif choice == "4":
            system_check()
        elif choice == "5":
            run_speech()
        elif choice == "6":
            fuse_face_speech()
        elif choice == "q":
            break
        else:
            print("■ Not a valid option.")
if __name__ == "__main__":
    main()
----- END FILE: app\strokeai_demo.py -----
----- BEGIN FILE: modules\__init__.py -----
----- END FILE: modules\__init__.py -----
----- BEGIN FILE: modules\stroke\__init__.py -----
----- END FILE: modules\stroke\__init__.py -----
```

```
----- BEGIN FILE: modules\stroke\features\__init__.py -----
----- END FILE: modules\stroke\features\__init__.py -----
----- BEGIN FILE: modules\stroke\features\face_asymmetry.py -----
# ------------------------------------------------------------
#  Open NeuroHealth Framework © 2025 Parameshwar
#  Author: Parameshwar  |  Version: 1.0 (Day 7 Build)
#  Licensed under the MIT License
#  This software is an original work developed for
#  neuro-diagnostic and research awareness applications.
# ------------------------------------------------------------
"""
StrokeAI - NetLag HUD + Face Asymmetry
Keys:
  r = start/stop recording (auto-saves CSV on stop)
  c = force-save CSV anytime
  s = save averaged JSON
  q = quit
"""
import cv2, mediapipe as mp, numpy as np, uuid, os, csv, time, json
from datetime import datetime
from pathlib import Path
# ---------- Absolute Paths ----------
ROOT = Path(__file__).resolve().parents[3]
LOG_DIR = ROOT / "data" / "logs"
EXP_DIR = ROOT / "data" / "exports"
LOG_DIR.mkdir(parents=True, exist_ok=True)
EXP_DIR.mkdir(parents=True, exist_ok=True)
mp_drawing = mp.solutions.drawing_utils
mp_face_mesh = mp.solutions.face_mesh
# Symmetric landmark pairs
PAIRS = [(61, 291), (133, 362), (33, 263)]
L_EYE_OUT, R_EYE_OUT = 33, 263
def _pt(landmarks, idx, w, h):
    lm = landmarks[idx]
    return np.array([lm.x * w, lm.y * h], dtype=np.float32)
def compute_asymmetry_from_landmarks(landmarks, image_shape):
    h, w = image_shape[:2]
    eps = 1e-6
    try:
```

```
        iod = np.linalg.norm(
            _pt(landmarks, R_EYE_OUT, w, h) - _pt(landmarks, L_EYE_OUT, w, h)
        ) + eps
    except Exception:
        return 1.0, {"error": "incomplete landmarks"}
    l_eye_ref = _pt(landmarks, L_EYE_OUT, w, h)
    r_eye_ref = _pt(landmarks, R_EYE_OUT, w, h)
    diffs = []
    for L_idx, R_idx in PAIRS:
        try:
            Lp = _pt(landmarks, L_idx, w, h)
            Rp = _pt(landmarks, R_idx, w, h)
            L_vert = abs(Lp[1] - l_eye_ref[1]) / iod
            R_vert = abs(Rp[1] - r_eye_ref[1]) / iod
            diffs.append(abs(L_vert - R_vert))
        except Exception:
            continue
    ai = float(np.mean(diffs)) if diffs else 1.0
    return ai, {"pairs": len(diffs), "inter_ocular": float(iod)}
def _save_csv(ai_log):
    """Write ai_log to CSV (absolute path). ai_log = [(t_abs, ai_or_nan), ...]"""
    if not ai_log:
        print("■■ Empty log, not writing CSV.")
        return None
    csv_path = LOG_DIR / f"face_ai_session_{datetime.now().strftime('%Y%m%d-%H%M%S')}.csv"
    base_t = ai_log[0][0]
    with csv_path.open("w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow(["t_sec", "ai"])
        for t_abs, ai in ai_log:
            w.writerow([t_abs - base_t, "" if np.isnan(ai) else ai])
    print(f"■ Saved CSV → {csv_path}")
    return csv_path
def _save_avg_json(ai_log):
    valid = [v for _, v in ai_log if not np.isnan(v)]
    if not valid:
        print("■■ No valid AI samples to average.")
        return None
    avg_ai = float(np.mean(valid))
    record = {
        "id": str(uuid.uuid4()),
        "module": "stroke",
        "timestamp": datetime.utcnow().isoformat() + "Z",
        "signals": {"face_asymmetry_avg": avg_ai, "samples": len(valid)},
```

```
        "device": "webcam",
        "notes": "live capture average",
    }
    json_path = EXP_DIR / "neuro_unit_face_ai_avg.json"
    with json_path.open("w", encoding="utf-8") as f:
        json.dump(record, f, indent=2)
    print(f"■ Saved averaged AI JSON → {json_path}")
    return json_path
def run_webcam():
    cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
    if not cap.isOpened():
        cap = cv2.VideoCapture(0)
    if not cap.isOpened():
        print("■ Could not open webcam.")
        return
    print("■ Keys: r=start/stop CSV, c=force-save CSV, s=save avg JSON, q=quit")
    recording = False
    ai_log = []
    start_time = None
    last_time = time.time()
    fps = 0.0
    with mp_face_mesh.FaceMesh(
        static_image_mode=False,
        max_num_faces=1,
        refine_landmarks=True,
        min_detection_confidence=0.5,
        min_tracking_confidence=0.5,
    ) as face_mesh:
        while True:
            ok, frame = cap.read()
            if not ok:
                print("■■ cap.read() failed")
                break
            # FPS (EWMA)
            now = time.time()
            dt = now - last_time
            if dt > 0:
                fps = 0.9 * fps + 0.1 * (1.0 / dt)
            last_time = now
            frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            results = face_mesh.process(frame_rgb)
            current_ai = np.nan
            if results.multi_face_landmarks:
                fl = results.multi_face_landmarks[0]
                mp_drawing.draw_landmarks(
```

```
                frame,
                fl,
                mp_face_mesh.FACEMESH_TESSELATION,
                landmark_drawing_spec=None,
                connection_drawing_spec=mp_drawing.DrawingSpec(
                    thickness=1, circle_radius=1
                ),
            )
            ai, _ = compute_asymmetry_from_landmarks(fl.landmark, frame.shape)
            current_ai = float(ai)
            cv2.putText(
                frame,
                f"AI: {current_ai:.3f}",
                (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.7,
                (0, 255, 0),
                2,
            )
        else:
            cv2.putText(
                frame,
                "Face not detected",
                (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.7,
                (0, 0, 255),
                2,
            )
        # HUD overlay
        cv2.putText(
            frame,
            f"FPS: {fps:.1f}",
            (10, 60),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.6,
            (255, 255, 0),
            2,
        )
        if recording:
            cv2.putText(
                frame,
                "REC ●",
                (540, 30),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.8,
                (0, 0, 255),
                2,
            )
            ai_log.append((time.time(), current_ai))
        cv2.imshow("StrokeAI - NetLag HUD", frame)
        key = cv2.waitKey(1) & 0xFF
```

```
            # --- Keys ---
            if key == ord("r"):
                if not recording:
                    ai_log = []
                    start_time = time.time()
                    recording = True
                    print("■ Recording started...")
                else:
                    recording = False
                    duration = time.time() - start_time
                    print(
                        f"■ Recording stopped. {len(ai_log)} samples, {duration:.1f}s"
                    )
                    _save_csv(ai_log)  # ■ auto-save CSV when stopped
            elif key == ord("c"):
                if ai_log:
                    _save_csv(ai_log)
                else:
                    print("■■ No samples yet to save.")
            elif key == ord("s"):
                _save_avg_json(ai_log)
            elif key == ord("q"):
                break
    cap.release()
    cv2.destroyAllWindows()
if __name__ == "__main__":
    print("■ ROOT =", ROOT)
    print("■ LOG_DIR =", LOG_DIR)
    print("■ EXP_DIR =", EXP_DIR)
    run_webcam()
----- END FILE: modules\stroke\features\face_asymmetry.py -----
----- BEGIN FILE: modules\stroke\features\speech_rate.py -----
# ------------------------------------------------------------
#  Open NeuroHealth Framework © 2025 Parameshwar
#  Author: Parameshwar  |  Version: 1.0 (Day 7 Build)
#  Licensed under the MIT License
#  This software is an original work developed for
#  neuro-diagnostic and research awareness applications.
# ------------------------------------------------------------
# modules/stroke/features/speech_rate.py
"""
Speech rate (syllables/sec) + voiced ratio prototype
```

```
- Lists mic devices (--list)
- Records audio with chosen device
- Adjustable sensitivity (energy/ZCR percentiles)
- Saves WAV + Neuro Unit JSON
"""
from datetime import datetime
import uuid, os, json, argparse
import numpy as np
import sounddevice as sd
from scipy.signal import medfilt
from scipy.io import wavfile
# ----------------------------
# Helpers
# ----------------------------
def list_devices():
    print("■■ Input devices:")
    devs = sd.query_devices()
    for i, d in enumerate(devs):
        if d.get("max_input_channels", 0) > 0:
            print(f" {i:2d}: {d['name']}  (in={d['max_input_channels']})")
    print("\nTip: pick an index with nonzero 'in='.\n")
def ensure_dirs():
    os.makedirs("data/exports", exist_ok=True)
    os.makedirs("data/logs", exist_ok=True)
def record(duration, fs, channels, device=None):
    print(f"■■ Recording {duration:.1f}s @ {fs} Hz, ch={channels} (device={device}) … Speak natu
rally.")
    x = sd.rec(int(duration*fs), samplerate=fs, channels=channels, dtype="float32", device=devic
e)
    sd.wait()
    x = x.squeeze()  # (N,) if mono, (N,C) -> (N,)
    return x
def frame_signal(x, fs, win_ms=30, hop_ms=10):
    win = int(fs*win_ms/1000)
    hop = int(fs*hop_ms/1000)
    if win < 3: win = 3
    if hop < 1: hop = 1
    n = max(0, (len(x)-win)//hop + 1)
    if n <= 0:
        return np.empty((0, win), dtype=x.dtype), win, hop
    frames = np.stack([x[i*hop:i*hop+win] for i in range(n)], axis=0)
    return frames, win, hop
def estimate_syllables_per_sec(x, fs, energy_pctl=40, zcr_pctl=60, agc=False, debug=False):
    """
    energy_pctl: lower = more sensitive to quiet speech (default 40)
    zcr_pctl:    higher = more tolerant to consonants/noise (default 60)
    agc:         normalize to unit RMS before analysis
    """
    x = np.asarray(x, dtype=np.float32)
    if x.size == 0:
```

```
        return 0.0, 0.0, {"frames": 0, "voiced_frames": 0, "mean_abs_amp": 0.0}
    mean_abs_amp = float(np.mean(np.abs(x)))
    if agc and mean_abs_amp > 0:
        x = x / max(np.sqrt(np.mean(x**2)), 1e-6)
    frames, win, hop = frame_signal(x, fs)
    if frames.size == 0:
        return 0.0, 0.0, {"frames": 0, "voiced_frames": 0, "mean_abs_amp": mean_abs_amp}
    # Features
    energy = (frames**2).mean(axis=1)
    # zero-cross rate per frame
    signs = np.sign(frames)
    zcr = (np.abs(np.diff(signs, axis=1)) > 0).mean(axis=1)
    # Smooth with median filter (kernel must be odd)
    k = 7 if len(energy) >= 7 else max(3, (len(energy)//2)*2+1)
    energy_s = medfilt(energy, kernel_size=k)
    zcr_s = medfilt(zcr, kernel_size=k)
    # Thresholds
    e_th = np.percentile(energy_s, np.clip(energy_pctl, 1, 99))
    z_th = np.percentile(zcr_s, np.clip(zcr_pctl, 1, 99))
    # Voiced heuristic
    voiced = (energy_s > e_th) & (zcr_s < z_th)
    # Count 'islands' (rising edges of voiced)
    if voiced.size:
        islands = int(np.logical_and(voiced, np.concatenate([[False], ~voiced[:-1]])).sum())
    else:
        islands = 0
    dur_sec = len(x)/fs
    voiced_ratio = float(voiced.mean()) if voiced.size else 0.0
    sps = float(islands / max(dur_sec, 1e-6))
    details = {
        "frames": int(len(energy)),
        "voiced_frames": int(voiced.sum()),
        "mean_abs_amp": mean_abs_amp,
        "energy_pctl": energy_pctl,
        "zcr_pctl": zcr_pctl,
        "islands": islands,
        "duration_sec": dur_sec
    }
    if debug:
        print(f"Debug: frames={len(energy)}, voiced={voiced.sum()} ({voiced_ratio:.2f})")
        print(f"Debug: islands={islands}, duration={dur_sec:.2f}, sps={sps:.2f}")
        print(f"Debug: mean|x|={mean_abs_amp:.4f}, e_th={e_th:.6g}, z_th={z_th:.6g}")
    return sps, voiced_ratio, details
```

```
def save_wav(x, fs, path="data/exports/last_speech.wav"):
    ensure_dirs()
    # clip to int16 for wav
    y = np.clip(x, -1.0, 1.0)
    wavfile.write(path, fs, (y * 32767).astype(np.int16))
    print(f"■ Saved WAV → {path}")
    return path
def save_neuro_unit_speech(sps, voiced_ratio, duration, fs, device_label, path="data/exports/neu
ro_unit_speech_rate.json"):
    ensure_dirs()
    record = {
        "id": str(uuid.uuid4()),
        "module": "stroke",
        "timestamp": datetime.utcnow().isoformat() + "Z",
        "signals": {
            "speech_syllables_per_sec": float(sps),
            "voiced_ratio": float(voiced_ratio),
            "duration_sec": float(duration),
            "fs_hz": int(fs)
        },
        "device": device_label,
        "notes": "speech rate prototype"
    }
    with open(path, "w", encoding="utf-8") as f:
        json.dump(record, f, indent=2)
    print(f"■ Saved Speech Neuro Unit → {path}")
    return path
# ----------------------------
# CLI
# ----------------------------
def main():
    ap = argparse.ArgumentParser(description="Speech rate (syll/sec) + voiced ratio")
    ap.add_argument("--list", action="store_true", help="List input devices and exit")
    ap.add_argument("--device", type=int, default=None, help="Input device index")
    ap.add_argument("--fs", type=int, default=48000, help="Sample rate (Hz)")
    ap.add_argument("--channels", type=int, default=1, help="Channels (1=mono)")
    ap.add_argument("--duration", type=float, default=5.0, help="Seconds to record")
    ap.add_argument("--energy-pctl", type=float, default=40.0, help="Energy percentile threshold
 (lower = more sensitive)")
    ap.add_argument("--zcr-pctl", type=float, default=60.0, help="ZCR percentile threshold (high
er = more tolerant)")
    ap.add_argument("--agc", action="store_true", help="Normalize input (auto-gain) before analy
sis")
    ap.add_argument("--debug", action="store_true", help="Print debug stats")
    args = ap.parse_args()
    if args.list:
        list_devices()
        return
    # Validate settings before recording
    try:
        sd.check_input_settings(device=args.device, samplerate=args.fs, channels=args.channels)
    except Exception as e:
        print("■ Settings invalid:", e)
        print("Tip: run  python tools\\audio_probe.py  to find a working (device/fs/ch).")
```

```
        print("     or run with  --list  to see available input devices.")
        return
    # Record
    x = record(duration=args.duration, fs=args.fs, channels=args.channels, device=args.device)
    print(f"■ Mean |x| amplitude: {np.abs(x).mean():.5f}")
    save_wav(x, args.fs, "data/exports/last_speech.wav")
    # Analyze
    sps, vr, det = estimate_syllables_per_sec(
        x, args.fs,
        energy_pctl=args.energy_pctl,
        zcr_pctl=args.zcr_pctl,
        agc=args.agc,
        debug=args.debug
    )
    print(f"■ Estimated speech rate: {sps:.2f} syll/sec  (voiced ratio: {vr:.2f})")
    # Persist Neuro Unit
    label = f"device={args.device}" if args.device is not None else "default-mic"
    save_neuro_unit_speech(sps, vr, args.duration, args.fs, label)
if __name__ == "__main__":
    main()
----- END FILE: modules\stroke\features\speech_rate.py -----
----- BEGIN FILE: modules\stroke\fusion\fuse_face_speech.py -----
# ------------------------------------------------------------
#  Open NeuroHealth Framework © 2025 Parameshwar
#  Author: Parameshwar  |  Version: 1.0 (Day 7 Build)
#  Licensed under the MIT License
#  This software is an original work developed for
#  neuro-diagnostic and research awareness applications.
# ------------------------------------------------------------
# modules/stroke/fusion/fuse_face_speech.py
# Read latest Neuro Unit files for face & speech; output fused risk JSON.
# modules/stroke/fusion/fuse_face_speech.py
"""
Fuse Face Asymmetry (AI) + Speech Rate into a simple stroke-risk prototype.
- Loads latest Face AI avg JSON and Speech rate JSON
- Computes risks and overall risk (0..1)
- Shows a small bar chart (Face, Speech, Overall)
- Saves data/exports/stroke_risk_fused.json
"""
import os, json, glob, math
from datetime import datetime
import numpy as np
import matplotlib.pyplot as plt
```

```
FACE_GLOB = "data/exports/neuro_unit_face_ai_avg.json"       # we overwrite this file each save
SPEECH_GLOB = "data/exports/neuro_unit_speech_rate.json"     # we overwrite this file each save
OUT_JSON = "data/exports/stroke_risk_fused.json"
def _load_json(path):
    if not os.path.isfile(path):
        return None
    with open(path, "r", encoding="utf-8") as f:
        try:
            return json.load(f)
        except Exception:
            return None
def _clamp(x, lo=0.0, hi=1.0):
    return max(lo, min(hi, float(x)))
def _face_risk(face_ai):
    """
    Map asymmetry index (lower is better) to risk 0..1.
    Rough heuristic:
      <= 0.02 -> ~0 risk (very symmetric)
      0.02..0.12 -> rising risk
      >= 0.12 -> ~1 risk
    """
    return _clamp((face_ai - 0.02) / (0.12 - 0.02))
def _speech_risk(sps, voiced_ratio):
    """
    Map speech rate (syll/sec) + voiced ratio to risk 0..1.
    Heuristic:
      Normal conversational ~3–7 syll/sec.
      If sps < 3, risk grows; also penalize very low voiced_ratio.
    """
    # risk from slow speech
    r_sps = _clamp((3.0 - float(sps)) / 2.0)  # 3 -> 0, 1 -> 1 (cap 0..1)
    # risk from low voicing
    r_vr = _clamp((0.5 - float(voiced_ratio)) / 0.25)  # <=0.25 -> 1, >=0.5 -> 0
    # combine (soft OR)
    return _clamp(0.7 * r_sps + 0.3 * r_vr)
def _grade(r):
    if r < 0.3:
        return "LOW", "Proceed normally"
    if r < 0.6:
        return "MODERATE", "Observe carefully / retest"
    return "HIGH", "Seek medical attention if symptoms persist"
def fuse_and_plot():
    # --- load inputs ---
    face = _load_json(FACE_GLOB)
    speech = _load_json(SPEECH_GLOB)
    if face is None:
        print("■■ Face JSON not found. Run menu [1] first to save averaged AI JSON.")
```

```
    if speech is None:
        print("■■ Speech JSON not found. Run menu [5] to record speech JSON.")
    if face is None or speech is None:
        return
    # --- read values ---
    face_ai = float(face["signals"].get("face_asymmetry_avg", np.nan))
    sps = float(speech["signals"].get("speech_syllables_per_sec", np.nan))
    voiced_ratio = float(speech["signals"].get("voiced_ratio", np.nan))
    if np.isnan(face_ai) or np.isnan(sps) or np.isnan(voiced_ratio):
        print("■■ Missing numeric values in inputs.")
        return
    # --- compute risks ---
    r_face = _face_risk(face_ai)
    r_speech = _speech_risk(sps, voiced_ratio)
    # overall (weight face a bit higher in this prototype)
    overall = _clamp(0.6 * r_face + 0.4 * r_speech)
    label, suggestion = _grade(overall)
    # --- print summary ---
    print("\n=== StrokeAI Fusion Summary ===")
    print(f" Face AI avg:     {face_ai:.3f}  -> face risk {r_face:.2f}")
    print(f" Speech rate sps: {sps:.2f}  (voiced {voiced_ratio:.2f}) -> speech risk {r_speech:.2
f}")
    print(f" Overall risk:    {overall:.2f}  [{label}]  — {suggestion}")
    # --- save JSON ---
    os.makedirs(os.path.dirname(OUT_JSON), exist_ok=True)
    out = {
        "id": face.get("id", "") or speech.get("id", ""),
        "module": "stroke",
        "timestamp": datetime.utcnow().isoformat() + "Z",
        "inputs": {
            "face_json": FACE_GLOB,
            "speech_json": SPEECH_GLOB
        },
        "signals": {
            "risk_face": r_face,
            "risk_speech": r_speech,
            "risk_overall": overall,
            "face_ai_avg": face_ai,
            "speech_sps": sps,
            "voiced_ratio": voiced_ratio
        },
        "notes": "Prototype fusion; not a medical device"
    }
    with open(OUT_JSON, "w", encoding="utf-8") as f:
        json.dump(out, f, indent=2)
    print(f"■ Saved fused risk JSON → {OUT_JSON}")
    # --- plot small bar chart ---
    labels = ["Face", "Speech", "Overall"]
    vals = [r_face, r_speech, overall]
```

```
    plt.figure(figsize=(5, 3.2))
    plt.bar(labels, vals)          # (no custom colors/styles)
    plt.ylim(0, 1)
    plt.ylabel("Risk (0–1)")
    plt.title(f"StrokeAI Fusion — Overall: {overall:.2f} [{label}]")
    plt.grid(True, axis="y", alpha=0.3)
    plt.tight_layout()
    plt.show()
if __name__ == "__main__":
    fuse_and_plot()
----- END FILE: modules\stroke\fusion\fuse_face_speech.py -----
----- BEGIN FILE: tools\audio_probe.py -----
# ------------------------------------------------------------
#  Open NeuroHealth Framework © 2025 Parameshwar
#  Author: Parameshwar  |  Version: 1.0 (Day 7 Build)
#  Licensed under the MIT License
#  This software is an original work developed for
#  neuro-diagnostic and research awareness applications.
# ------------------------------------------------------------
import sounddevice as sd
CANDS_FS = [16000, 24000, 44100, 48000]
CANDS_CH = [1, 2]
def main():
    devs = sd.query_devices()
    print("■■ Input devices:")
    idx_map = []
    for i, d in enumerate(devs):
        if d["max_input_channels"] > 0:
            print(f"{i:2d}: {d['name']}  (in={d['max_input_channels']})  hostapi={sd.query_hosta
pis()[d['hostapi']]['name']}")
            idx_map.append(i)
    pick = input("\nEnter device index (or Enter for default): ").strip()
    device = int(pick) if pick else None
    print("\n■ Probing formats...")
    for fs in CANDS_FS:
        for ch in CANDS_CH:
            try:
                sd.check_input_settings(device=device, samplerate=fs, channels=ch)
                print(f"■ OK: fs={fs} Hz, channels={ch}")
            except Exception as e:
                print(f"■ fs={fs}, ch={ch} -> {e}")
    # If something passes, try 3s capture/playback with the first OK combo
    for fs in CANDS_FS:
```

```
        for ch in CANDS_CH:
            try:
                sd.check_input_settings(device=device, samplerate=fs, channels=ch)
                print(f"\n■■ Recording 3s @ {fs} Hz, ch={ch} ...")
                x = sd.rec(int(3*fs), samplerate=fs, channels=ch, dtype="float32", device=device
)
                sd.wait()
                print("■■ Playback...")
                sd.play(x, fs); sd.wait()
                print("■ Worked.")
                return
            except Exception:
                continue
    print("\n■■ No working combo found. Try a different device index.")
if __name__ == "__main__":
    main()
----- END FILE: tools\audio_probe.py -----
----- BEGIN FILE: tools\audio_check.py -----
# ------------------------------------------------------------
#  Open NeuroHealth Framework © 2025 Parameshwar
#  Author: Parameshwar  |  Version: 1.0 (Day 7 Build)
#  Licensed under the MIT License
#  This software is an original work developed for
#  neuro-diagnostic and research awareness applications.
# ------------------------------------------------------------
import sounddevice as sd
import numpy as np
def list_inputs():
    print("■■ Input devices:")
    for i, dev in enumerate(sd.query_devices()):
        if dev["max_input_channels"] > 0:
            print(f"{i:2d}: {dev['name']}  (in={dev['max_input_channels']})")
def record_play(seconds=3, fs=16000, device=None):
    print(f"\n■■ Recording {seconds}s @ {fs} Hz ...")
    x = sd.rec(int(seconds*fs), samplerate=fs, channels=1, dtype="float32", device=device)
    sd.wait()
    print("■■  Playing back...")
    sd.play(x, fs); sd.wait()
    print("■ Done")
    return x
if __name__ == "__main__":
    list_inputs()
    ans = input("\nEnter device index to test (or Enter for default): ").strip()
    dev = int(ans) if ans else None
    record_play(device=dev)
```

```
----- END FILE: tools\audio_check.py -----
----- BEGIN FILE: tools\netlag_hud_test.py -----
# ------------------------------------------------------------
#  Open NeuroHealth Framework Â© 2025 Parameshwar
#  Author: Parameshwar  |  Version: 1.0 (Day 7 Build)
#  Licensed under the MIT License
#  This software is an original work developed for
#  neuro-diagnostic and research awareness applications.
# ------------------------------------------------------------
import cv2, csv, os, time
from datetime import datetime
# Ensure output dir
os.makedirs("data/logs", exist_ok=True)
print("ðŸŽ›ï¸ NetLag HUD Test")
print("Keys: r=start/stop CSV logging, q=quit")
# try DSHOW first (most reliable on Windows), fallback to default
cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
if not cap.isOpened():
    cap = cv2.VideoCapture(0)
if not cap.isOpened():
    print("â■Œ Could not open webcam.")
    raise SystemExit
rec = False
log = []
t0 = None
fps = 0.0
last_t = time.time()
while True:
    ok, frame = cap.read()
    if not ok:
        print("âš ï¸ cap.read() failed")
        break
    # FPS estimate (EWMA)
    now = time.time()
    dt = now - last_t
    if dt > 0:
        fps = 0.9 * fps + 0.1 * (1.0 / dt)
    last_t = now
    # HUD: FPS + REC
    hud_color = (0, 255, 255)
    cv2.putText(frame, f"FPS: {fps:.1f}", (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, hud_color, 2)
    if rec:
        cv2.putText(frame, "REC â—■", (540, 30),
```

```
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0,0,255), 2)
    # If recording, append a simple signal (frame index over time)
    if rec:
        log.append((time.time(), len(log)))
    cv2.imshow("NetLag HUD (FPS + REC)", frame)
    key = cv2.waitKey(1) & 0xFF
    if key == ord('r'):
        if not rec:
            log.clear()
            t0 = time.time()
            rec = True
            print("ðŸŽ¥ Recording startedâ€¦")
        else:
            rec = False
            dur = time.time() - t0 if t0 else 0
            print(f"ðŸ›‘ Recording stopped. samples={len(log)}, dur={dur:.1f}s")
            if log:
                csv_path = f"data/logs/netlag_{datetime.now().strftime('%Y%m%d-%H%M%S')}.csv"
                with open(csv_path, "w", newline="", encoding="utf-8") as f:
                    w = csv.writer(f)
                    w.writerow(["t_sec", "value"])
                    base = log[0][0]
                    for t, v in log:
                        w.writerow([t - base, v])
                print("âœ… CSV saved â†’", csv_path)
            else:
                print("âš ï¸ No samples collected; CSV not written.")
    elif key == ord('q'):
        print("ðŸ‘‹ Quit.")
        break
cap.release()
cv2.destroyAllWindows()
----- END FILE: tools\netlag_hud_test.py -----
----- BEGIN FILE: tools\key_csv_test.py -----
# ------------------------------------------------------------
#  Open NeuroHealth Framework Â© 2025 Parameshwar
#  Author: Parameshwar  |  Version: 1.0 (Day 7 Build)
#  Licensed under the MIT License
#  This software is an original work developed for
#  neuro-diagnostic and research awareness applications.
# ------------------------------------------------------------
import cv2, csv, os, time
from datetime import datetime
import numpy as np
```

```
# Make sure folders exist
os.makedirs("data/logs", exist_ok=True)
print("ðŸ“¹ KEY CSV TEST")
print("Press 'r' to start/stop, 'q' to quit")
recording = False
log = []
start_t = None
# Create blank image for testing
frame = np.ones((240, 320, 3), dtype=np.uint8) * 255
while True:
    cv2.putText(frame, f"REC={recording}  samples={len(log)}", (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255) if not recording else (0, 255, 0), 2)
    cv2.imshow("KEY CSV TEST", frame)
    key = cv2.waitKey(1) & 0xFF
    if key == ord('r'):
        if not recording:
            log.clear()
            start_t = time.time()
            recording = True
            print("ðŸŽ¥ Recording startedâ€¦")
        else:
            recording = False
            duration = time.time() - start_t if start_t else 0
            print(f"ðŸ›‘ Recording stopped. {len(log)} samples, {duration:.1f}s")
            if log:
                csv_path = f"data/logs/key_test_{datetime.now().strftime('%Y%m%d-%H%M%S')}.csv"
                with open(csv_path, "w", newline="", encoding="utf-8") as f:
                    writer = csv.writer(f)
                    writer.writerow(["t_sec", "value"])
                    t0 = log[0][0]
                    for t, v in log:
                        writer.writerow([t - t0, v])
                print(f"âœ… CSV saved â†’ {csv_path}")
            else:
                print("âš ï¸ No samples collected")
    elif key == ord('q'):
        print("ðŸ‘‹ Exiting.")
        break
    if recording:
        log.append((time.time(), len(log)))  # fake data stream
cv2.destroyAllWindows()
----- END FILE: tools\key_csv_test.py -----
```